

PEAS (Performance Measure, Environment, Actuators, Sensors):

1. Performance Measure:

A rational agent will choose actions that maximize performance measure.

2. Environment:

- Fully Observable (vs partially observable):

An agent's sensors give it access to the complete state of the environment at each point in time.

- Single Agent (vs multi-agent):

An agent operating by itself in an environment.

- Deterministic (vs stochastic): 有状态分布

The next state of the environment is completely determined by the current state and the action executed by the agent.

- Strategic:

The environment is also dependent on the actions of other agents

- Episodic (vs sequential):

Each decision is independent. The agent just looks at the current situation and acts, with no memory of past actions needed.

- Static (vs dynamic): 你不想, 世界在变

The environment is unchanged while an agent is deliberating.

- Semi-dynamic:

The environment itself does not change with the passage of time, but the agent's performance score does.

- Discrete (vs continuous):

A limited number of distinct, clearly defined percepts and actions.

The structure of Agents:

- Simple reflex agent: Maps percepts directly to actions via condition-action rules. No state, no memory.
- Goal-based agent: Maintains state, simulates future outcomes, selects actions that satisfy a goal condition.
- Utility-based agent: Extends goal-based by assigning a utility score to states; maximizes expected utility. 找期望效用最大的行动
- Learning agent: Improves performance element over time via critic feedback and exploratory actions.

Uninformed Search algorithms:

We do not know any information of how close a state is to the goal.

BFS	Expand shallowest node first	Queue
UCS	Expand lowest path cost	Priority Queue
DFS	Expand deepest node first	Stack
DLS	DFS with depth limit L	Stack
IDS	DLS repeated with L = 0, 1, 2, ...	Stack with depth

Informed Search algorithms:

We have a clue on how close a state is to the goal.

- Heuristic $h(n)$: Estimated cost from n to the goal.
- Admissible: $0 \leq h(n) \leq h^*(n)$, $h(goal) = 0$. where $h^*(n)$ is the optimal path cost to reach the goal state from n .
- Consistent: For every node n , every successor n' of n generated by any action a , $h(n) \leq c(n, a, n') + h(n')$. Consistency implies admissibility.
- Dominance: If $h_1(n) \geq h_2(n)$ for all n , then h_1 dominates h_2 .

A*	Expand nodes with lowest cost $f(n) = g(n) + h(n)$. Where $g(n)$ is cost to n and $h(n)$ is the chosen heuristic.	Priority Queue
----	--	----------------

Algo	Complete	Optimal	Time / Space
BFS	Yes	Yes, if all costs are the same.	Exponential
UCS	Yes, if edge costs are positive	Yes, if edge costs are positive	Exponential
DFS	Yes, if with visited memory. Else No	No	Exponential
DLS	Yes, if there's a solution within limit of steps. Else No	No, if used with DFS.	Polynomial (if used with DFS)
IDS	Yes, if BFS	Yes, if BFS	Exponential
A* with v	Yes, if UCS and consistent	Yes, if UCS and consistent	Exponential
A* without v	Yes, if UCS and admissible	Yes, if UCS and admissible	Exponential

Local Search:

- Simple Strategy: Traverse the state space by considering only the best locally reachable state:
- 1. Start at a (random) position in a state space.
- 2. Consider neighboring states of current state. (Move to "better" neighboring state, while "forgetting" the other neighboring states.)
- 3. The solution is the final state after a certain number of steps.
- Typically, incomplete and suboptimal.
- Any-time property: longer runtime, (often) better solution.

Hill climbing algorithm:

current_state = initial_state

while True:

best_successor = a highest-eval successor state of current_state

if eval(best_successor) <= eval(current_state):

return current_state

current_state = best_successor

- Global Maximum: The overall highest point or solution across the entire state space that represents the optimal solution.
- Local Maximum: A point in the state space that is higher than its immediate neighbors, but there may be a higher value elsewhere.
- Shoulder: A region in the state space where the objective function has a flat area.

Minimax algorithm and Alpha-Beta Pruning:

- Minimax computes the optimal strategy for both players, guaranteeing the best achievable outcome for each, assuming both play optimally.
- Alpha-beta pruning does not alter the minimax value and the optimal move computed for the root node.

def alpha_beta_search(state):

(action, v) = max_value(state, -∞, ∞)

return action

def max_value(state, α, β):

if is_terminal(state): return utility(state)

v = -∞

for next_state in expand(state):

v = max(v, min_value(next_state, α, β))

α = max(α, v)

if v >= β: return v

return v

def min_value(state, α, β):

if is_terminal(state): return utility(state)

v = ∞

for next_state in expand(state):

v = min(v, max_value(next_state, α, β))

β = min(β, v)

if v <= α: return v

return v

Performance measure:

[True/False] [Positive/Negative]:

The prediction is (Positive/Negative), and it is [True/False].

- Accuracy = (TP + TN) / (TP + FN + FP + TN)
- Precision = TP / (TP + FP)
- Recall = TP / (TP + FN)
- F1 score = 2 / (1/P + 1/R)

Decision Tree:

- For any finite set of consistent examples with discrete (categorical) features and a finite set of label classes, there exists a decision tree that is consistent with (i.e., perfectly classifies) the examples.

Base case:

1. No data available: Follow default value.

2. Same decision for all rows: Follow that decision.

3. Ran out of attributes: Follow the majority.

Pruning:

1. Max depth: Start with depth 0.

2. Min-sample leaves.

Linear Regression:

- Mean Square Error (MSE) Loss: $J_{MSE}(w) = \frac{1}{2n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)})^2$
- Variants of Gradient Descent:
- 1. Batch Gradient Descent: Consider all training examples.
- 2. Mini-batch Gradient Descent: Consider a subset of training examples at a time. Noisy gradient might help escape local minima/plateau.
- 3. Stochastic Gradient Descent (SGD): Select one training sample at a time. Noisier gradient. Cheapest (fastest) per iteration.
- Normalization: Solution to feature with different scales.

Min-max scaling

$$x'_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

Scales the features to be within [0, 1].

It is also common to scale the features to be within [-1, 1].

Standardization

$$w_j \leftarrow w_j - \frac{1}{n} \sum_{i=1}^n (w_j^T x^{(i)} - y^{(i)}) x_j^{(i)}$$

$$w_j \leftarrow w_j - \frac{1}{n} \frac{\sum_{i=1}^n (w_j^T x^{(i)} - y^{(i)}) x_j^{(i)}}{\sum_{i=1}^n x_j^{(i)2}}$$

	Gradient Descent	Normal Equation
Need to choose learning rate γ	Yes	No
Iteration(s)	Many	None, closed-form solution
Large number of features d ?	No problem	Slow, $(X^T X)^{-1} \rightarrow O(d^3)$
Feature scaling?	May be necessary	Not necessary
Constraints	-	$X^T X$ needs to be invertible

Logistic Regression:

- Sigmoid Function (aka logistic function): $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- Logistic model: $h_w(x) = \sigma(w_0 x_0 + w_1 x_1 + w_2 x_2 + \dots + w_d x_d) = \sigma(w^T x)$

Binary Cross-Entropy (BCE) loss:

$$J(w) = -\frac{1}{n} \sum_{i=1}^n (y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$$

Multi-class Classification:

	One-vs-One	One-vs-All
Binary Classifier	For every pair of classes.	For each class, treating all other classes as a single combined class.
Number of binary classifiers	$C(K, 2) = \frac{K(K-1)}{2}$	K
Prediction	Each classifier votes for a class, and the class with the most votes is selected.	The classifier with the highest probability output (confidence score) determines the class.
Example:	Classifier 1: Distinguish between class 1 and 2. Classifier 2: Distinguish between class 1 and 3. Classifier 3: Distinguish between class 2 and 3.	Classifier 1: Distinguish class 1 and not-1 (2, 3). Classifier 2: Distinguish class 2 and not-2 (1, 3). Classifier 3: Distinguish class 3 and not-3 (1, 2).

Multi-label Classification Common Techniques:

- 1. Binary Relevance: Train a separate model for each label.
- 2. Classifier Chain: Like binary relevance but feed predictions of previous labels as features for subsequent labels.
- 3. Label Powerset: Treat each unique label-set as a single multiclass label.

Regularization

Common regularizers:

- L_1 -norm (Manhattan norm)

$$R_{L_1}(w) = \|w\|_1 = \sum_{i=1}^d |w_i|$$

- L_2 -norm (Euclidean norm)

$$R_{L_2}(w) = \|w\|_2 = \sqrt{\sum_{i=1}^d w_i^2}$$

In practice, we simplify it to $\frac{1}{2} \sum_{i=1}^d w_i^2$

For lasso regression (MSE + L_1 -norm):

$$J(w) = J_{MSE}(w) + \lambda R_{L_1}(w)$$

$$\frac{\partial J(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w_j^T x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda \frac{\partial |w_j|}{\partial w_j}$$

$$\frac{\partial J(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w_j^T x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

- For any $\lambda > 0$, the L_2 -norm of the L_2 regularized weight vector is strictly smaller than the non-regularized weight vector, assuming the solution is non-zero.
- For any $\lambda > 0$, ridge regression (linear regression with L_2 regularization) has a unique, closed-form solution given by $w = (X^T X + \lambda I)^{-1} X^T Y$.

如果 $\lambda=0$, 没有正则化。

① L2 strength, 100% 在 A 范围内 = weights are too small to capture patterns
② prioritize min weights over fitting them. underfitting

regularization $\rightarrow$ model variance $\rightarrow$ less sensitivity to noise

* CNN Receptive field

$$r_i = f_{i-1} + (kernel_i - 1) \times j_{i-1}$$

$$j_i = j_{i-1} \times stride_i \quad \# \text{第 } i \text{ 层的卷积 stride}$$

Primal: $h_w(x) = \sum_{j=1}^n w_j x_j$ (d)

Dual: $h_w(x) = \sum_{j=1}^n \alpha_j x_j$ (n)

- If a feature has non-zero weight, the L1 regularizer will penalize the feature with a constant penalty of λ (in terms of gradient).
- For a feature to be included in the model, its contribution to reducing the MSE loss must be exactly equal to the regularization penalty λ . If a feature's contribution is less than the penalty λ , the optimization will prefer to set its weight to **exactly zero** rather than pay the L1 penalty, inducing sparsity.

Qual Models and Kernel Method: $h_w(x) \Rightarrow n \times d$

Representer Theorem (Restricted, very informal):
For a model parameterized with a weight vector w , in special cases the best weight vector w that minimizes a (regularized) loss over a training set of vectors $x(i)$ can be expressed as:

where α is a vector of real numbers $\alpha_1, \alpha_2, \dots, \alpha_n$ of dimension n .

Example: logistic regression with L2 regularization and Support Vector Machines.

As consequence, linear model can be expressed using the α vector as follows

The model:

- $h_w(x)$ is called the **primal formulation** of the model.
- $h_\alpha(x)$ is called the **dual formulation** of the model, that **min** the loss is a **Optimal model: primal = dual**.

Kernel Function: For any feature mapping function ϕ , the function $k(u, v) = \langle \phi(u), \phi(v) \rangle$ for any vectors u and v , is called a kernel function, or kernel. Suppose that you use the transformed feature $\phi(x)$ as input, your model becomes:

Kernel Trick: Simply compute kernel between $x^{(i)}$ and x , without having to expand $\phi(x)$ explicitly.

Kernel Machine: A model that uses kernel trick.

Hyperplane Classification Model

- Definition:** The zero-offset hyperplane is defined by a normal vector w and contains all vectors u for which $w^T u = 0$.
- The only difference between this and logistic model is **This uses sign function and logistic model uses sigmoid function.**

Hyperplane Classification Primal & Dual Model:

Support Vector Machine (SVM): "Optimal" hyperplane classification model. Computes the best decision boundary.

Objective (idea): Maximize the distance between the closest datapoints to the hyperplane.

Neuron Model: A neuron model computes a weighted sum of input features and passes the result through an activation function.

Here, $w_0 \dots w_d$ are weights and $x_0 \dots x_d$ are input features, where d is the dimension of the input features. g represents the activation function.

Common Activation Functions

Sigmoid: $g(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$

Identity: $g(x) = x$

Tanh: $g(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

ReLU: $g(x) = \max(0, x)$

Leaky ReLU: $g(x) = \max(\alpha x, x)$

Neuron Model: A neuron model computes a weighted sum of input features and passes the result through an activation function.

FC layer: $z = \sum_{j=1}^n w_j x_j$

Activation function: $y = g(z)$

Loss function: $L(y, y') = \frac{1}{2} \sum_{i=1}^n (y_i - y'_i)^2$

Test/train:

- ① test/train
- ② 用 training set
- ③ 计算 μ 和 σ
- ④ 用 μ 和 σ 归一化训练集
- ⑤ 训练模型
- ⑥ 用同样的 μ 和 σ 归一化测试集
- ⑦ test model

Feed-forward Neural Networks (FFNNs):

- Definition:** A special case of neural networks where the computation graph has designated input and output and there are no cycles.
- Layer:** A layer in a neural network is a parameterized map $w: X_{in} \rightarrow X_{out}$ that takes an input from a domain X_{in} (e.g., vector) and produces an output in a domain X_{out} .
- Deep Learning:** Neural networks with more than one hidden layer.

Fully-connected Neural Networks (FCNNs):

- Definition:** A special case of feed-forward neural networks that are composed of sequential fully-connected layers.
- Fully-connected (FC) Layer:** A layer that is composed of parallel neuron models. All input nodes are connected to all neuron models.
- Layer:** A group of nodes that are applied in parallel.
- Depth:** The number of layers (except input layer) applied sequentially.
- Width:** The number of neuron models in a layer.

Multi-class classification problem:

- SoftMax function:** computes categorical probability distribution vector from a vector of real numbers. Given a vector z , the probability of belonging to class i is:

Categorical Cross-Entropy (CCE) Loss:

Example: Let $y = [1, 0, 0]$ and $\hat{y} = [0.8, 0.1, 0.1]$.

Convolutional Neural Networks (CNNs): 只处理图像信息, 不处理其他信息

The setup:

- Input shape before padding: $I_H \times I_W$
- Kernel shape: $K \times K$
- Padding: P
- Stride: S

Output Width: $1 + \frac{I_W + 2P - K}{S}$

Output Height: $1 + \frac{I_H + 2P - K}{S}$

Kernel / Filter:

Receptive field: 当前像素的一个神经元, 追溯回去, 对应原图多大区域

Learnable parameters: (Kernel size + 1) * number of kernels

Convolutional Layer:

- Pre-activation:** $Z_i = X * W_i + b_i$
- Post-activation:** $(A_i)_{k,q} = g((Z_i)_{k,q})$

The activation function is applied element-by-element to the pre-activation output.

Pooling Layer:

- Common Pooling Functions:** Max-Pool, Average-Pool
- Dimensionality Rules:**
 - Spatial downsampling:** The sliding window operates across the height and width of the feature map.
 - Channel independence:** Each feature map is processed completely separately from the others.

Recurrent Neural Networks (RNNs):

- Token:** The smallest unit of data that a model processes.
- Special Token:** <SOS> (Start of Sequence), <EOS> (End of Sequence)
- Hidden State:** A hidden state ($h^{(t)}$) is a vector that acts as model's memory of the sequence processed so far.
- Bidirectional RNN Layer:** A single Bidirectional RNN layer contains two independent RNN layers (separate weight matrices) operating side-by-side on the exact same input sequence.
- The Bidirectional RNN layer produces a single unified output for each time step:** $h^{(t)} = (h_{forward}^{(t)}, h_{backward}^{(t)})$
- Stacked RNN:** The RNN architecture where multiple RNN layers are stacked on top of each other. Each RNN layer has its own hidden state.

Unsupervised Learning:

- Pretext Task: Rotation Prediction.** Transformation: For each image $x^{(i)}$, generate rotated versions $x^{(i,j)}$, $j \in \{0, 1, 2, 3\}$. $j=0$: Rotate 0°; $j=1$: Rotate 90°; $j=2$: Rotate 180°; $j=3$: Rotate 270°
- Task:** Train a model to solve a 4-class rotation prediction task.
- Input:** The image $x^{(i,j)}$ rotated 0°
- Target:** The rotation class label $y^{(i,j)} = [1, 0, 0, 0]^T$
- Pretext Task: Self-Supervised Contrastive Learning.** Positive Pair: Two different augmented views of the same image. Negative Pair: Augmented views of a different (random) images.
- The model learns:** Similar features for augmented versions of the same image
- Pretext Task: Image Inpainting.** Task: "Mask" random patches from an input image. Force the model to predict the missing pixel values.
- The model learns:** It understands the content of the image.
- Pretext Task: Next-Token Prediction.** Task: Train the model to predict the next token given previous tokens. The label (next token) comes directly from the data.
- The model learns:** grammar, syntax, real-world factual knowledge.

Neural Network Architecture:

Input Layer: $x^{(1)} \in \mathbb{R}^d$

Hidden Layer: $h^{(1)} \in \mathbb{R}^d$

Output Layer: $h^{(2)} \in \mathbb{R}^d$

Weight Matrices: $W^{(1)} \in \mathbb{R}^{d \times d}$, $W^{(2)} \in \mathbb{R}^{d \times d}$

Bias Vectors: $b^{(1)} \in \mathbb{R}^d$, $b^{(2)} \in \mathbb{R}^d$

Activation Function: g

Loss Function: $L(y, y') = \frac{1}{2} \sum_{i=1}^n (y_i - y'_i)^2$

Neuro

$h^{(t)} = f(x^{(t)}, h^{(t-1)}) = g((W^{(xh)})^T x^{(t)} + (W^{(hh)})^T h^{(t-1)} + b)$

Inputs & Outputs:

- $x^{(t)} \in \mathbb{R}^d$: Current input token representation
- $h^{(t-1)} \in \mathbb{R}^d$: Previous hidden state
- $h^{(t)} \in \mathbb{R}^d$: Current hidden state

Learned Parameters: Applied in all time steps

- $W^{(xh)} \in \mathbb{R}^{d \times d}$: Weight matrix for the input token
- $W^{(hh)} \in \mathbb{R}^{d \times d}$: Weight matrix for the previous hidden state
- $b \in \mathbb{R}^d$: Bias vector (Optional)

The Function:

- g : Activation function (e.g., tanh)

Teacher Forcing: A training strategy used for autoregressive sequence prediction models. The input for the next time step ($t+1$) is the true target of the current time step (t), not the model's own prediction.

Attention Neural Networks: (有时) 自动包含 positional encoding

- Self-Attention Mechanism:** Query: The search query. "What am I looking for?" Key: The keyword. "This is the kind of information I have." Value: The actual content. "This is the actual data I can contribute."
- The Linear Project:** Query: $Q = W^q X$, Key: $K = W^k X$, Value: $V = W^v X$
- Input:** $x^{(i)} \in \mathbb{R}^d$, Matrices: $W^q \in \mathbb{R}^{d \times d_q}$, $W^k \in \mathbb{R}^{d \times d_k}$, $W^v \in \mathbb{R}^{d \times d_v}$
- Rule:** $d_q = d_k = d_v = d$
- The Master Equation:**

Positional Encoding:

To inject positional "timestamps" directly into the token representations, we can add a positional vector. It must fulfill the properties below:

- Unique:** Each position should have a distinct encoding.
- Consistent:** The encoding for a specific location should be the same.
- Bounded:** The values must stay within a reasonable range.

For a given position t and dimension index k :

The Sinusoidal Solution:

If k is even: $PE_{[k]}^{(t)} = \sin(\frac{t}{10000^{2k/d}})$

If k is odd: $PE_{[k]}^{(t)} = \cos(\frac{t}{10000^{2k/d}})$

Masked Self-Attention: To prevent each position from seeing future tokens, we need to apply a mask to attention scores.

Cross-Attention Mechanism: In many-to-many salutation, it allows the decoder to check the encoder's output and find relevant information.

The linear project:

Query: $Q = W^q D$ Key: $K = W^k E$ Value: $V = W^v E$

Dimensionality:

- Weight matrix:** $W^q \in \mathbb{R}^{d_q \times d_{dec}}$; $W^k \in \mathbb{R}^{d_k \times d_{enc}}$; $W^v \in \mathbb{R}^{d_v \times d_{enc}}$
- Sequences:** $D \in \mathbb{R}^{d_{dec} \times T_y}$; $E \in \mathbb{R}^{d_{enc} \times T_x}$
- Q, K, V:** $Q \in \mathbb{R}^{d_q \times T_y}$; $K \in \mathbb{R}^{d_k \times T_x}$; $V \in \mathbb{R}^{d_v \times T_x}$; $d_q = d_k = d_v = d$

Unsupervised Learning:

- Pretext Task: Rotation Prediction.** Transformation: For each image $x^{(i)}$, generate rotated versions $x^{(i,j)}$, $j \in \{0, 1, 2, 3\}$. $j=0$: Rotate 0°; $j=1$: Rotate 90°; $j=2$: Rotate 180°; $j=3$: Rotate 270°
- Task:** Train a model to solve a 4-class rotation prediction task.
- Input:** The image $x^{(i,j)}$ rotated 0°
- Target:** The rotation class label $y^{(i,j)} = [1, 0, 0, 0]^T$
- Pretext Task: Self-Supervised Contrastive Learning.** Positive Pair: Two different augmented views of the same image. Negative Pair: Augmented views of a different (random) images.
- The model learns:** Similar features for augmented versions of the same image
- Pretext Task: Image Inpainting.** Task: "Mask" random patches from an input image. Force the model to predict the missing pixel values.
- The model learns:** It understands the content of the image.
- Pretext Task: Next-Token Prediction.** Task: Train the model to predict the next token given previous tokens. The label (next token) comes directly from the data.
- The model learns:** grammar, syntax, real-world factual knowledge.

Neural Network Architecture:

Input Layer: $x^{(1)} \in \mathbb{R}^d$

Hidden Layer: $h^{(1)} \in \mathbb{R}^d$

Output Layer: $h^{(2)} \in \mathbb{R}^d$

Weight Matrices: $W^{(1)} \in \mathbb{R}^{d \times d}$, $W^{(2)} \in \mathbb{R}^{d \times d}$

Bias Vectors: $b^{(1)} \in \mathbb{R}^d$, $b^{(2)} \in \mathbb{R}^d$

Activation Function: g

Loss Function: $L(y, y') = \frac{1}{2} \sum_{i=1}^n (y_i - y'_i)^2$

标准二分类 setup: output layer: 1 neuron (Sigmoid) + loss f: BCE

标准多分类 setup: output layer: n neuron (softmax) + loss f: Cross Entropy

标准回归 setup: output layer: 1 neuron (Linear) + loss f: MSE

标准生成式 setup: output layer: n neuron (Gaussian) + loss f: KL Divergence